

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>
#include <limits.h>

#define MAX_HIST 100
#define BUF_SIZE 512
#define STATE_FILE "shell_state.txt"

char *history[MAX_HIST];
int history_count = 0;

// Save current session state and command history to file
void save_state() {
    FILE *fp = fopen(STATE_FILE, "w");
    if (!fp) {
        perror("Failed to save state file");
        return;
    }
    for (int i = 0; i < history_count; i++) {
        fprintf(fp, "%s\n", history[i]);
    }
    fclose(fp);
    printf("[State Saved] Written %d history entries to %s\n", history_count, STATE_FILE);
}

// Add entry to in-memory history table
void add_history(const char *cmd) {
    if (strlen(cmd) == 0) return;
    if (history_count < MAX_HIST) {
        history[history_count++] = strdup(cmd);
    }
}

// Clean up allocated resources before exiting
void cleanup_resources() {
    save_state();
    for (int i = 0; i < history_count; i++) {
        free(history[i]);
    }
    printf("[Cleanup] Freed history buffers and shut down gracefully.\n");
}

// Validate variable names (Must start with letter/underscore and contain alphanumeric/underscore)
int is_valid_var_name(const char *name) {
    if (!name || (!isalpha(name[0]) && name[0] != '_')) return 0;
    for (int i = 1; name[i] != '\0'; i++) {
        if (!isalnum(name[i]) && name[i] != '_') return 0;
    }
    return 1;
}

// Parse export syntax: export VAR=VALUE or export VAR
int handle_export(const char *arg) {
    const char *eq_pos = strchr(arg, '=');
    if (eq_pos == NULL) {
        // Syntax: export VAR
        if (!is_valid_var_name(arg)) {
            printf("export: '%s': not a valid identifier\n", arg);
            return 1;
        }
    }
}
```

```

        return 0; // Variable name valid, nothing to update
    }

    // Split VAR and VALUE
    size_t var_len = eq_pos - arg;
    char var_name[128];
    strncpy(var_name, arg, var_len);
    var_name[var_len] = '\0';

    const char *var_val = eq_pos + 1;

    // Remove optional surrounding quotes from value
    char clean_val[BUF_SIZE];
    size_t val_len = strlen(var_val);
    if ((var_val[0] == '"' && var_val[val_len - 1] == '"') ||
        (var_val[0] == '\'' && var_val[val_len - 1] == '\'')) {
        strncpy(clean_val, var_val + 1, val_len - 2);
        clean_val[val_len - 2] = '\0';
    } else {
        strcpy(clean_val, var_val);
    }

    if (!is_valid_var_name(var_name)) {
        printf("export: '%s': not a valid identifier\n", var_name);
        return 1;
    }

    // Update environment variable (overwrite if exists)
    if (setenv(var_name, clean_val, 1) == 0) {
        printf("[Export] Successfully set %s=%s\n", var_name, clean_val);
        return 0;
    } else {
        perror("setenv failed");
        return 1;
    }
}

```

```

// Launch child process to verify environment variable propagation
void test_child_export(const char *var_name) {
    pid_t pid = fork();
    if (pid == 0) {
        char *val = getenv(var_name);
        printf("[Child Process PID %d] Injected %s = %s\n", getpid(), var_name, val ? val : "(null)");
        exit(0);
    } else if (pid > 0) {
        waitpid(pid, NULL, 0);
    }
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        // Retrieve current directory and display path prompt
        char cwd[PATH_MAX];
        if (getcwd(cwd, sizeof(cwd)) == NULL) strcpy(cwd, "shell");

        printf("\n[%s] > ", cwd);
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline

        // Process Exit Requests
        if (strncmp(input_buf, "exit", 4) == 0 && (input_buf[4] == '\0' || input_buf[4] == ' ')) {
            int exit_code = 0;
            if (strlen(input_buf) > 5) {
                exit_code = atoi(input_buf + 5);
            }
            cleanup_resources();
            printf("Exiting shell with status code %d\n", exit_code);
            exit(exit_code);
        }

        if (strlen(input_buf) == 0) continue;

        add_history(input_buf);

        // Handle Export Built-in
        if (strncmp(input_buf, "export ", 7) == 0) {
            handle_export(input_buf + 7);
        } else if (strncmp(input_buf, "testchild ", 10) == 0) {
            test_child_export(input_buf + 10);
        } else if (strncmp(input_buf, "pwd", 4) == 0) {
            printf("%s\n", cwd);
        } else {
            printf("Command recorded in history. (Use 'export VAR=VAL', 'testchild VAR', 'pwd', or 'exit [code]')\n");
        }
    }

    cleanup_resources();
    return 0;
}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano export_state.c
abhis@MIKHAIL-37:~$ gcc -g -Wall export_state.c -o export_state
abhis@MIKHAIL-37:~$ ./export_state

[/home/abhis] > export MY_VAR=HelloLinux
[Export] Successfully set MY_VAR=HelloLinux

[/home/abhis] > export 123INVALID=Value
export: '123INVALID': not a valid identifier

[/home/abhis] > testchild MY_VAR
[Child Process PID 1535] Injected MY_VAR = HelloLinux

[/home/abhis] > exit 0
[State Saved] Written 3 history entries to shell_state.txt
[Cleanup] Freed history buffers and shut down gracefully.
Exiting shell with status code 0
abhis@MIKHAIL-37:~$ valgrind --leak-check=full ./export_state
==1547== Memcheck, a memory error detector
==1547== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1547== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1547== Command: ./export_state
==1547==

[/home/abhis] > exit
[State Saved] Written 0 history entries to shell_state.txt
[Cleanup] Freed history buffers and shut down gracefully.
Exiting shell with status code 0
==1547==
==1547== HEAP SUMMARY:
==1547==    in use at exit: 0 bytes in 0 blocks
==1547==   total heap usage: 3 allocs, 3 frees, 2,520 bytes allocated
==1547==
==1547== All heap blocks were freed -- no leaks are possible
==1547==
==1547== For lists of detected and suppressed errors, rerun with: -s
==1547== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```